Contexto Morphus - Para desenvolvedores e IAs

Projeto Morphus

O projeto Morphus é dividido em dois aplicativos uma para o front end e outro para o back end, Morphus possui um banco de dados postgtres e esta programado para subir e rodar em containesr docker. os dois aplicativos são:

- mpx-frontend e mpx-backend => As duas aplicações dazem parte de um único repositório no github (Monorepositorio) e
- Hoje nos temos uma VPS rodando ubntu lunux e docker, porém apenas a aplicação frontend e um banco de dados postgres foram publicado lá.
- Temos também um domínio registrado e apontando para o frontend https://www.morphus.me
- Tambem thems uma conta no https://www.auth0.com ele faz hoje tofda nossa parte de autenticação ee validação de usuários, ao chegar no frontend por exemplo, o usuário e redirecionado para a pagina de login deles, mas esta pagina esta toda customizada para o morphus
- Também ja está confgurado fazer login com o google. existe no google cloud uma aplicação chamada morphus configurada.

- Morphus frontend (mpx-frontend)
  - É um aplicativo Anguluar rodando hoje na versão 22.2.2 com node na versão 24.22.0 versão.
  - Não possui suporte para SSR (Server Side Render), roda somente no browser
  - Utiliza uma biblioteva chamada BuilderKit clicando aqui você pode ver a documentação oficial

- BuilderKit é uma biblioteca com vários componentes simples, componentes avancados, 4 trmplates prontos e 27 blocos de construção prontos para serem adaptados e utilizados
- A diferença enre blocos de contrução e componentes écomplexidade e as funcionalidades de cada um, clocos de construção as vezes controem paginas inteiras sozinhos, e components sao alsudos em partes da pagina ou de outrs componentes ou blocos
- nNa raiz do projeto existe uma pasta chamada .builderkit dentro dela você verá a pasta templates com os 4 templates, a pasta blocks com todo os blocos de contrucao e uma outra pasta chamada builderkit que possui uma min estrutura de pastas com componentes e funcionaidades que estão em praticamente em todos os projetos / templates
- Todos os templates são basicamente templates de administração, uma barra lateral resposniva com menu, uma barra no topo com informações de usuário entre outras informações e bootoes de ação e um content no centro aonde as paginas filhas são rendenizadas.

- Estrtura do projeto

FOLDER_STRUCTURE.md 98 kB

No arquievo acima, que tambem está na raiz do projeto, está a estrutura completa da nossa aplicação hoje.

O que é importante neste momento?

> Existem no projeto, 5 arquivos com instruções e habilidades para os agentes de IA

- O primeiro arquivo que esta na raiz e fala dos dois projetos é este que voce está lendo agora MORPHUS_CONTEXT -> este arquivo possui um breve detalhamento do contexto da aplicção MORPHUS
- O segundo arquivo está na raiz da aplicação angular mpx-frontend Ele se chama AGENTS.md e ele diz basicamente que as UI (User Interfaces) do projeto frontend são construidas usando blocos de contrucão e indica como aonde estão as Skills para os agentes saberem configurar atualizar e montar os blocos.
- O terceiro arquivo esta dentro da pasta ./builderkit/blocks/INDEX.md,

  esse arquivo tem descrições sobre cada bloco e como chegar até ele e como utilizá-lo

  O quarto e o quinto arquivo esta dentro da pasta ./agents/skills/builderkit/ na raiz da aplication frontend e possem habilidades para os agentes

  - guideline.md => baseicamente descrive as diretrizes a serem seguidas para a construção do design da aplicacao frontend, fala sobre estilização, themas, componentes, blocos paginas e como isso tudo deve se encaixar.
  - SKILLS.md => Este arquivo detalha aiinda mais toda a biblioteca builderkit

- Sobre o resto da estrutura de mpx-frontend

Voce pode ver na estrutura do projeto mpx-fronend, que na raiz, tempos alem dos arquivos padões,

- **Um Dockerfile =>** Esse aquivo está ~~deprecate.~~ quando este projeto foi criado pela primeira vez, ele usava SSR para rendenizar os htmls em um server node js e era esse arquivo que montava a imagem desse server rodando node em um container docker, porém desconitua,os a parte do SSR, estava apresentando muita instabilidade na autenticação
- **docker-compose.yml =>** Este arquvo builda e instala a aplicao frontend e o banco de dados postgres
- **CaddyFile =>** o CaddyFile cria a imagem para um outr container com a funcionaliade de gerar um certificado https para o dominio

Drnyto da pasta sorce temos

- **index.html => aonde tudo é reindenizado**
- **Styles => dentro da pasta styles, temos os arquivos .css do projeto**
  - **base => na pasta base estão os arquivos que estilizam a base do aplicativo, body, htm, h1, h2 tipografias etc...**
  - **components => aqui os components por exemplo compojnents do angular material, tailwindcss são sobrescritos reestilizados ou atravez de aplicação de novos estilos ou definindo valores de variaveis css. somente alguns componentes sao modificados aqui**
  - **material => ja na psta material, praticamente todos os componentes do angular material são modificaos para o builderkit**

- **tailwindcss** => aqui temos funcionalidades de thema do tailwindcss
- **thirt-part** => aqui só existe uma classe css e ela se referencia aos icones Lucidde Icons
- **styles.css** => se voce reparou tudo esta em subpastas, menos este arquivo que edta sozinho na razi da pasta styles, é ele quem está registrao no angular.json como sendo nosso css global e ele esta aqui nesta pasta recebendo as importacoes de todos arquivos dentro das subpastas, se criarmos uma pasta para morphus por exemplo e criarmos dentro dela um morphus.css para que ele seja jogado para dentro da pagina da aplicacao basta fazer uma referencia dele dentro de styles.css (@import './morphus/morphus.css' por exemplo.
- **enviroument** => nesta pasta estao as variaveis do sistema, por exemplo, aqui estao definidas as variaveis que sao utilizadas para chamada da bibiliotec de auth0
- **components** => aque nesta pasta devem ficar os componentes globais, hoje dentro de components temos apenas a pasta builderkit com alguns components quando criarmos nosso primeiro complente global será criada a pasta morphus e dentro dela noddos componentes

- **A pasta apps** essa pasta explica bem como o ecossistema morphus vai funcionar, alem da pasta core que é utilizada para armazenar funcionaliades, uteis, tipos tamto para o builderkit como para morphus teemos 3 outras pastas. Essas 3 pastas são o coração do Morphus e ela subdivide o morphus em 3 sub aplicações
  - **A aplicação morphus =>** Essa aplicação é o carro chefe do negocio, ela é um painel adminitrativo, que vao regir as outras duas apps,
  - **A aplicação security =>** Essa aplicação cuida da segurançao do sistema, telas de login, cadastro recuperação de senha chamadas a auth0 etre outras coisas são responsabilidade de security
  - **a aplicação website =>** é aqui que o usuário vai ver, vai navegar, somente aqui, ele nao vai conhecer uma app chamada security, e so vai cair dentro de home em website e navegar pelas funcionalidades do morphus por ali, obedecendo, sem ele saber o que security e morphus criaram como regras.

FOLDER_STRUCTURE.md 98 kB

- **Morphus backend (mpx-backend)**

Morphus backend roda em .Net Core verrsão **11.0.100-rc.2.26504.105** e segue o mais alto padrão de design partner a Clean Achetecture.

Ele é separado em 5 camadas e de forma muito clara e coerente cada uma sabe exatamente exerce o seu papel. As camadas são:

- Web Api -
  - Responsável por receber as requisições HTTP, Verificar a permissão desta requisição seguir com a requisição adiante, para a camada Application), receber e retornar o conjunto de dados solicitado.
  - É principal defesa de segurança do sistema morphus backend e para isso utiliza
  - Microsoft.AspNetCore.Authentication.JwtBearer versão (11.0.0-rc.1.26425.128), integrada com o site www.auth0.com e utilizando a bibilioteca oficial ela representa uma fortaleza.
  - a quem ela faz referência: Em teoria a clean archtecture permite que uma camade como a wenapi so referencie a camada Application, pois ela deve ser 100% desacoplada do banco de dados e das regras de negocio, porém é necessário que ela referencie todas as outras camadas, mas com um onjetivo simples e sussinto, por exemplo: como injetamos a dependência e inicializaos o contexto do banco de dados sem que a webapi saiba quem é o contexto? então ele precisa conhecer os objetos da camada de infrastructure. porám esta é a unica função que ela pode fazer com as 3 camadas as quais ela nao tem permssão de conhecer, somente registro de dependências, qualquer outro acesso é extritammente proibido, a não ser para a camada Application que é o fluxo normall da web api, ela somente tem direito de acessar a camada Applicationj e chamar seus métodos

- Application
  - A camada Appliocarion é reponsável pelos nossos hanflresr servicos, casos de uso...
  - O que é um handler / Servico: São métodos que como o nome ja diz, tratam as requisições da Api tralizando alguma operação sobre elas, encaminhand-as para proxima camada e esperando o resultado e é ela quem efetua boa parte da logivca envolvida nessas requisições, a camada Application pode fazer muitas coisas antes de devolver o resultado da requisição para a api
  - A quem ela faz referência: A camada Application apenas faz referência à Camada de Domain.
- Domain
  - Domains contem as entidades do projeto, os enumns, asinterfaces do sistema, insterface por exemplo como a do do repositório genérico e da interface de transação de banco de dados as interfaces de cada entidade
  - A quem ela faz referência: A NINGUEM, Domain é sozinha, c# puro, e não depende de ninguem, é dessa forma que ela executa as regras de nêncio do sistema, sem precisar de ninguem para isso
- InfraStructure
  - A camada de Infrastructure, é a responável pelo banco de dados, consulta, inserção, atualização enntre outras, no nosso caso ela utiliza EntityFramework core para isso.

    - Alem disso ela implementa muitas interfaces da camada de Domain, por exemplo os repositórios de acesso a dados de cada entidad. Esses repositórios são classes que implementam os metodos das interfaces do dominio e esses métodos vao ao banco de dados
    - É ela que contem o dbContext (MorphusDbContext), as Migrations entre outras.
    - A quem ela faz referência, A camada Infrastructure, depende apenas da camada de Application, Mas como a camada Application faz referância a camada de Domains, os dados de Domains acabam chegando ate a camada de Infrastructure, e isso não é nada proibido pela clean achitecture, pelo contrário, Infrastructure, precisa conhecer as entidades por exemplo, para registralas no contexto ddo banco de dados, essa referância cruzada se chama (transitively gets Domain),
- CrossCutting
    - A camada CrossCuttin e responsável por unicamente, registrar todas, sem excessão todas as injeções de dependência do ssistema, no final da contas ela é referenciada no arquivo programs.cs da wenapi e la ela chama uma unica método que recebe o Web Builder como parametro e dentro deste método ela utiliza esse buiilder de referência para registrar as injeções de depensdância

- Porque isso é muito legal? Tudo fuica centralizado apenas em um unico arquivo, seu program.cs, que iria ter umas 200 linhas vai ter epans 1 referente a injeção de depenencia, fácil de dar manutenção, achar aonde estão os registros.
- A quem ela referencia, a todass as outras camadasmenus a webapi, pois é a wenapi que faz referencia a ela, para que ela chegue ate programs.cs

.

Essas duas apps, front end e back end irao se comunicar o tempo todo, font end, fazendo solicitacoes ao back end.

E o controle de usuários e aceso, este controle 90% nas mãos da institução auth0.com

Lá serão caastrados organizações e dentro de cada uma delas são cadastrados usuários, roled, prtmissões, e tudo isso vai imbutido dentro do JWT Token que eles enviam para nós quando um usuário é autenticado, so precisamos restringir os metodos da webapi com authorize e politicas de seguranca baseados daqueles valos cadstrados no auth0 e atribuidos au usuário.

Mas nao se engane, teremos um trabalho sim e bem grande pois no morphus um usuário pode pertencer a mais de uma organizacao ao logar no nosso sistema o usuário poderá ter que escolher em qual organização ele vai logar, alem isse teremos que implementar MultiTenant para termos um reponsável por todos o HTenant

O que é o Morphus

Morphus é um gestor de corporações e suas filiais, é um gestor de acessos e permmissões, no morphus uma corporação possui um autorelacionamentom pois ela pode ter n filiais abaixo delas, entao o mesmo usuário pode ter aceso a 4 filiais diferente por exemplo entao teremos que tratar essas funcionaliades que o auth0 nao trata, no nosso banco de dados.

O morphus é um sistema de entrada de nota fiscal,cadastro de produtos, casfastros em geral como naturaze de operação, tributação, contas a pagar, contaas a receber, importacao automatica de notas fiscais, conciliacao bancaria, Api de envio de emails atravez de templates html castrados no banco de dados, e outras funcionalidades que estao sendo pensadass ainda.

Algumas regras de padrão de desenvolvimento muito iimportantes

1. Repare que todos os componentes angular criados, tem no mínimo um .ts e um .html e possi a seguinte nomenclatura, app.component.tsapp.component.html e esse htnl é reeferenciado dentro do .ts
    i. sempre faca assim,
    ii. <nom-do-componente>.component.ts
    iii. <nom-do-componente>.component.htmls
    iv. Jamais coloque o html debtro do .ts na tag template, isso dificulta imensamente a leiura visualizacao,e manutencao
2. Dentro de uma pasta voê irá enccontra varias subpastas, ou varios arquivos, jamais misture subpasta e arquvos dentro de uma pasta, mesmo que seja um unico arquivo crie uma subpasta para ele. Iss porque futuramente podem aparecer novos arquvos irãos daque que ficaria sozinha ai vc teria que criar a sub pasta para colocar os dois irmão juntos e quebraria o codigo todo.

As regras acima são para aplicação no frontend. Isso porque no backend a cleananchitecture resolve muito a respeito de pasroes de desenvolvimento.